CSCI 551 - Group Project

Caleb Whitman, Anna Watson, Mark Harris
Gianforte School of Computing
Montana State University
Bozeman, MT 59717, USA

1. Introduction

A fundamental algorithmic problem on a tree structure is determining the least common ancestor (LCA). Given a rooted tree T, and two nodes, u and v, which node is the LCA? That is, which node is furthest from the root that is an ancestor of both u and v. The LCA problem has been thoroughly studied because it is utilized to solve other algorithmic problems. Therefore, a fast algorithm must be utilized for the LCA to keep subsequent algorithms performant.

Harel and Tarjan were the first to publish an algorithm which produces LCA queries in constant time after a linear pre-processing time on T. The paper, "Fast Algorithms for Finding the Nearest Common Ancestors", is often cited as a means to achieve optimal algorithms which utilize the LCA. Harel and Tarjan (1984).

Alternatively, Bender and Farach published "The LCA Problem Revisited". In this work, the authors proposed another LCA algorithm which also queries in constant time after a linear pre-processing time on T. This algorithm first performs a reduction and then provides an alternative solution to the LCA problem. Bender and Farach-Colton (2000). In this paper, we discuss the original Harel and Tarjan algorithm and follow it up with a discussion on the Bender and Farach algorithm. To conclude, we compare the two.

2. Fast Algorithms for Finding Nearest Common Ancestors

Harel and Tarjan's paper discusses a variety of data structures and queries for the LCA problem. We will focus on the basic LCA algorithm with static trees and dynamic queries.

2.1 Intuition: Complete Binary Trees

The intuition behind the algorithm is that the LCA problem has a $O(1)$ query time for complete, labeled binary trees. Suppose we are given a binary tree with labels based on the pre-order traversal of the tree. Given the labels of two nodes, u and v, the depth of the nearest common ancestor is $d = \lfloor log(u \oplus v) \rfloor$. Using d and u, we can then find the location of the LCA, which is given by $2^{d+1}\lfloor u/2^{d+1} \rfloor + 2^d$. Proofs are omitted.

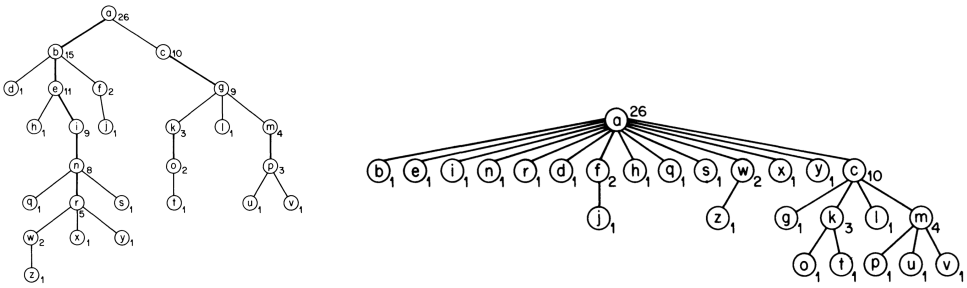
Thus, we use complete binary trees in order to speed up querying time. Two other tree structures, compressed trees and balanced binary trees, allow us to reduce our tree into a

complete binary tree. Using all of these trees plus a few auxiliary lookup structures, we obtain a $O(1)$ query time for any general tree.

This tree reduction construction is rather complex and has multiple interlocking pieces. The algorithm is so convoluted that some have labeled the algorithm as being "far too complicated to implement effectively" Bender and Farach-Colton (2000). Given this, we will present only a higher level description. The goal is to give an intuitive sense for how the algorithm functions.

2.2 Compressed Trees

A compressed tree shrinks down the original tree, allowing for faster operations and less storage space. We compress our original tree T by removing/compressing *heavy edges*. *Heavy edges* are any edges $u \to v$, such that $2size(u) > size(v)$ where $size(a)$ is the count of all descendants of a. Heavy edges often lie along long paths. By removing them, we eliminate long paths and shorten query times.



(a) Regular Tree (b) Compressed Tree

Figure 1: Comparison of Regular and Compressed Trees Harel and Tarjan (1984)

To find the LCA of T, we first find the LCA of the compressed tree C. Call it C_{LCA}. C_{LCA} exists nearby to the T_{LCA} and we can find T_{LCA} of C_{LCA} in constant time. See Harel and Tarjan for more details. Therefore, if we can quickly find an LCA of C, then we can find the equivalent LCA of T.

2.3 Balanced Binary Trees

Finding the LCA of C is unfortunately not trivial, and requires us to first represent C as a balanced binary tree B. The conversion from C to B adds auxiliary nodes into C until the tree is binary and balanced.

Once we have a balanced binary tree, we can efficiently compute any LCA for B (and subsequently C) using similar equations given for the LCA of the complete, balanced binary tree.

2.4 Plys

Finally, we divide C and B into three separate pieces based on height. We then pre-compute various LCA and height algorithms for each piece. This might seem like an extraneous step, but only because we have kept the discussion of the algorithm at a higher level. Keeping

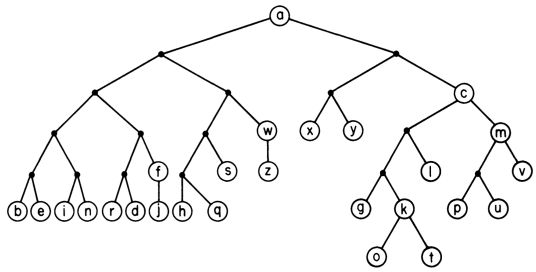


Figure 2: Balanced Binary TreeHarel and Tarjan (1984)

up with the trend, we only note that this pre-computation step is essential for a constant query time.

2.5 Complete Algorithm

Finally, we present the complete algorithm.

Algorithm 1 LCA

```
procedure LCA(Nodes u, v, Tree T)
    0. Construct compressed tree C and balanced binary tree B.
    1. Solve the LCA problem for B.
    2. Look up the LCA for C using the LCA for B.
    3. Use LCA for C to find the LCA for T
end procedure
```

3. The LCA Problem Revisited

Bender and Farach discovered that the LCA problem is equivalent to the RMQ problem. This greatly simplifies pre-processing and querying, leading to a far shorter algorithm.

3.1 LCA to RMQ

We first reduce the LCA problem to the RMQ problem. The RMQ problem is given a length n array of A numbers, and indices i, j, what is the index of the smallest number in the subarray from indices i to j?

1. Let array $E[1, ..., 2n-1]$ store nodes visited in a Euler Tour of the T where $E[i]$ is the label of the ith node visited in the Euler tour.

2. The level of a node is its distance from the root. Let array $L[1, ..., 2n-1]$ where $L[i]$ is the level of the node $E[i]$ from the Euler tour.

3. The representative of a node in the Euler tour is the index of the first occurrence of the node in the tour. That is, array $R[1, ..., n]$, where $R[i]$ is the index of the representative node i.

To then compute LCA from RMQ, we note the following:

1. The nodes in the Euler tour between u and v are $E[R[u], ..., R[v]]$.
2. The shallowest node in the sub-tour resides at index $RMQ_L(R[u], R[v])$.
3. Thus, the node located at position $E[RMQ_L(R[u], R[v])]$ is the output of the LCA of u and v.

Note that any two neighboring elements in RMQ_L differ by exactly 1. We will exploit this fact later.
We now look at solutions to the RMQ problem.

3.2 Sparse Table Method for Range Minimum Queries

Is there a way to reduce the pre-processing time and not sacrifice this favorable query time? Yes, but there is a trade-off between the pre-computation and querying times. Pre-computing the answers when the number of ranges is high leads to large computational time. However, if there are only a few ranges, the query time won't be O(1). We want to have n^2 total ranges, but enough ranges for querying in constant time.

Pre-processing:

For each index i, we compute RMQ_A for ranges starting at the index which has a size equal to a power of 2: 1, 2, ..., 2^k. Also, there is the added constraint that the ranges fit in A. There are $O(n \log n)$ ranges.

Querying:

The range we are querying can be represented as the union of two of the ranges discussed in the previous step, where k is large such that $2^k \leq i + 2^j - 1$. Then the range $[i, j]$ can be formed by joining two ranges $[i, i + 2^k - 1]$ and $[j - 2^k + 1, j]$ with overlapping. Total time is $O(1)$.

33	45	63	27	54	56	99	92
0	1	2	3	4	5	6	7

(a)

	2^0	2^1	2^2	2^3
0	33	33	27	27
1	45	45	27	
2	63	27	27	
3	27	27	27	
4	54	54	54	
5	56	56		
6	99	92		
7	92			

(b)

Figure 3: Sparse Table (R) of Array A (L)Lawson

So, the sparse table data structure for RMQ has a time complexity of $\langle O(nlog(n)), O(1)\rangle$ Fischer and Heun (2006).

3.3 $\mp$ RMQ Algorithm

Finally, we describe an algorithm that lowers the pre-processing step from $O(nlog(n))$ to $O(n)$. We take advantage of the fact that the Euler tour of the tree produces a $\mp$ RMQ data structure. The $\mp$ RMQ data structure has a smaller range of inputs than the generalized

RMQ data structure. In our pre-processing step, we store subsets of our input and use these later to produce our constant run time.

Pre-processing:

First, divide the $\mp$ RMQ array into blocks of size $log(n)/2$. Next, create the following four data structures:

1. A = minimum element in each block. We perform the sparse table pre-processing step on A to quickly answer queries over blocks.
2. I = index of minimum element.
3. R = difference array for each block. The difference array is created when you subtract each element from its following element. Due to the $\mp$ structure, there are a finite amount of possible difference arrays. For blocks of size $log(n)/2$, we get $2^{log(n)/2-1} = O(\sqrt(n))$ arrays. We store which array corresponds to each block.
4. H = table of all possible queries over all possible difference arrays.

	(0,1)	(0,2)	(0,3)	...
+1,+1,+1	0	0	0	
-1,-1,-1	1	2	3	
+1,-1,-1	0	0	3	
...				$\ddots$

Table 1: Example R table.

Querying:

Suppose we are given a query (i, j). For all full blocks in between i and j, use the sparse table algorithm to find the minimum element. For all partial blocks, look up the minimum value using R and H (since H stores all possible queries). All look ups are $O(1)$ time, completing the algorithm.

4. Analysis

Both algorithms pre-process the data in $O(n)$ time which allow us to perform $O(1)$ queries. The difference between each algorithm comes from the pre-processing data structures and methods. Harel and Tarjan recognized that complete binary trees can answer any LCA query in constant time. By creating a series of simpler interconnected trees, faster queries can be achieved on the original tree. The complex relationship between tree structures and the use of plys is complicated and largely unimplementable. Bender and Farach recognized that the LCA problem is equivalent to the $\mp$ RMQ problem. An optimal solution to the RMQ problem gives an equally fast solution to the LCA problem. Due to the simplicity of this data structure, this process can be easily and rapidly implemented.

Both algorithms use a variant of a lookup table which remains critical for the constant query time. Harel and Tarjan initialize arrays for the plys of their compressed and balanced

binary trees. These arrays allow a constant lookup time for LCA and height problems over these trees. Bender and Farach use a lookup table for all possible $\mp$ difference arrays in each block. Additionally, they utilized the sparse table data structure for constant lookup of full block queries.

Finally, Harel and Tarjan requires substantially more steps to implement than Bender and Farach, leading to a more unwieldy algorithm.

5. Conclusion

The Bender and Farach algorithm provides a simple, implementable solution to the LCA problem. While the Harel and Tarjan algorithm is classically cited as the first means to achieve linear preprocessing and constant time queries, it is too complex to be easily implemented Bender and Farach-Colton (2000). Bender and Farach's LCA solution should be utilized for any general LCA problem.

References

Michael A. Bender and Martin Farach-Colton. The lca problem revisited. *LATIN 2000: Theoretical Informatics*, pages 88–94, 2000. doi: https://doi.org/10.1007/10719839_9.

Johannes Fischer and Volker Heun. Theoretical and practical improvements on the rmq-problem, with applications to lca and lce. In Moshe Lewenstein and Gabriel Valiente, editors, *Combinatorial Pattern Matching*, pages 36–48, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg. ISBN 978-3-540-35461-1.

Dov Harel and Robert Endre Tarjan. Fast algorithms for finding the nearest common ancestors. *SIAM Journal on Computing*, 13(2), 1984. doi: https://epubs.siam.org/doi/pdf/10.1137/0213024.

James Lawson.